

Task 3

Software Security - Building Blocks

Reward: 15 Points

Mode: Project / Group work (max. 3 students)

Due: 15.06.2025 -- 14:15

Information

- **Remote Exercise Framework (REF)**

Before you start the task, you should read, understand, and follow the instructions for the *Remote Exercise Framework* provided in Moodle.

- **Group Work**

This task can be completed in groups. You may work in a group of up to 3 students (i.e., 1, 2, or 3 people), and your group selection is binding for the remainder of the semester. It is sufficient if one person from the group submits the solution (both the code in REF and a submission PDF in Moodle).

- **Deliverables**

All code submissions are made directly in REF; you can find more details in the REF guide on Moodle. All code must be sufficiently and meaningfully commented. Solutions to pen & paper tasks are uploaded as a single PDF file to Moodle.

Important: We reserve the right to check all submissions for plagiarism and take appropriate action.

Task 1: Crackme

a) Analyze an Unknown Program

Analyze the program `crackme`, for which the source code is not available. The program expects the user to enter a *secret key* consisting of five uppercase letters. Your task is to find the correct key using GDB analysis. Briefly explain how the program derives the password from the secret key and submit an input that results in the success message "*Key is valid :)*". The password should be saved to a file named `key.txt`.

Hints:

- The user input is verified in the function `verify_key`, which contains a `call <address>` instruction that calls the C function `strcmp`. You do not need to analyze that function.
- Within GDB, you can use `x/s <address>` to display strings.

b) Reconstructing C Code

Reverse engineer the `crackme` program. Understand its functionality and submit a C source file that replicates this behavior. Save your reconstructed code in `reconstructed.c`.

Task name for remote login: crackme

Task 2: System Calls and Shellcode

Modern operating systems use system calls (syscalls) as well-defined interfaces for applications to communicate with the kernel---for example, to read or write files. The Linux kernel, for instance, defines 384 system calls for the x86 architecture. These interactions are typically abstracted away from the programmer through high-level libraries like `libc`. However, when exploiting binaries, we prefer to skip this layer of abstraction. In this task, you will explore syscalls and how they can be used to exploit a binary.

a) Hello Kernel

Your first task is to create the file `/home/user/message` and write `hium!` to it. For this assembly task, you are not allowed to use the standard C library `libc`. That means the object files must be linked directly using the GNU linker `ld` instead of `gcc`, like this (or using `make`):

```
nasm -felf32 code.asm -o answer.o &&  
ld -m elf_i386 answer.o -o answer
```

This means you must not use a `main` function or any library functions. Instead, you must use syscalls. Submit your *commented* assembly source code and ensure your program does not terminate with a `segmentation fault`. You can find a list of the Linux syscalls here: https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md#x86-32_bit

Task name for remote login: syscalling

b) Shellcode

A key technique in binary exploitation is injecting code into the target program. This code often spawns a shell and is therefore called “shellcode”. In this task, you will write and assemble your own shellcode using the `execve` syscall to spawn the shell `/bin/dash`. Avoid null bytes and keep your code independent of specific memory addresses.

Submit both your *commented* source code and the raw bytes of your shellcode as a comment in the code (i.e., `SHELLCODE: "\x90\x90\x90..."`).

- Shellcoding 101: <https://rayoflightz.github.io/shellcoding/linux/x86/2018/11/15/Shellcoding-for-linux-on-x86.html>
- Assemble and link shellcode with:
`nasm -felf32 shellcode.asm -o x.o && ld -m elf_i386 x.o -o shellcode`
- You can check if `/bin/dash` was spawned using `ps -p "$$"`.

Task name for remote login: shellcode

Task 3: Smashing the Stack for Fun and Profit

Consider the following vulnerable program:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  /// DEBUG NOTE: admin_pw @ 0xABAD1DEA
6  char *admin_pw = "sup3r_s3cr3t_passw0rd!!11elf";
7
8  /// DEBUG NOTE: start of function @ 0xC001D00D
9  int authenticate(char *password){
10     // first pad the password to hopefully avoid timing attacks
11     char padded_password[24];
12     strcpy(padded_password, password);
13     // check the password
14     if (strcmp(padded_password, admin_pw) == 0) return 1;
15     return 0;
16 }
17
18 /// DEBUG NOTE: start of function @ 0xB0BAB0BA
19 void accepted(){
20     printf("Welcome to the admin console, trusted user!\n");
21 }
22
23 void rejected(){
24     printf("Go away! You smell: (\n");
25     exit(1);
26 }
27
28 /// DEBUG NOTE: argv[1] = 0xCAFEBAFE
29 int main(int argc, char *argv[]){
30     int authed = authenticate(argv[1]);
31     if (authed) accepted();
32     else rejected();
33     return 0;
34 }
```

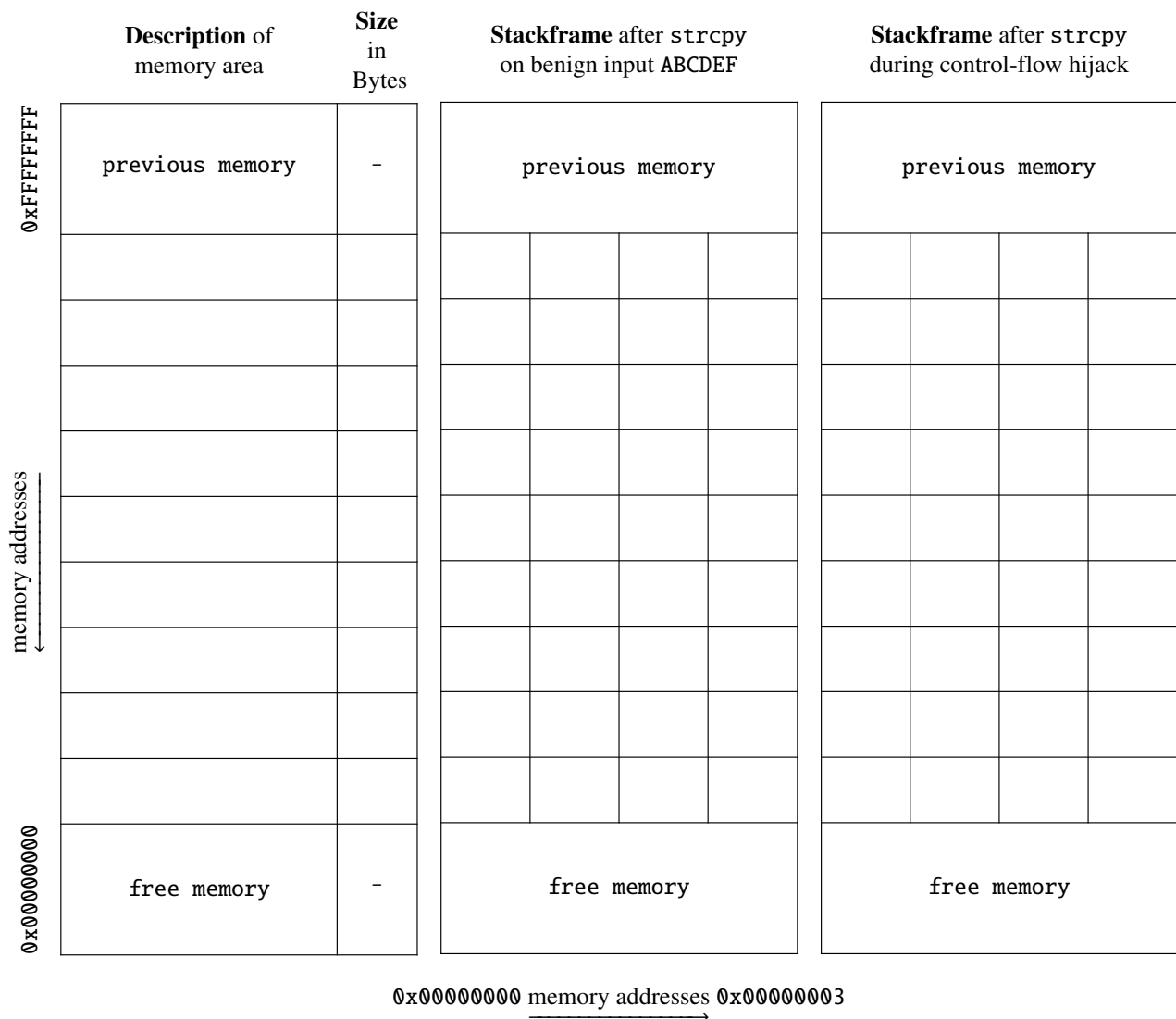
a) **Control Flow Manipulation**

Assume you have extracted various function and memory addresses during debugging (provided as `DEBUG NOTE` comments in the source). Your task is to use this information to build an exploit that overwrites the return address of the `authenticate` function, causing the program to continue executing in the `accepted()` function (without entering the correct password).

Consider the stack frame after the execution of `strcpy` in `authenticate(.)` for a “normal” run. Assume the program is called with the parameter `ABCDEF` and complete the left side of the template on the next page (description and stack frame). Note that C strings are null-terminated and `strcpy` copies until and including `0x00`. All undefined or unknown bytes/values can be marked with `'?'`.

Then construct a password (i.e., a command-line argument for the program) such that the `accepted(.)` function is executed (without using the real password). If padding bytes are needed, use the letter `A`. What does the stack frame look like after `strcpy` is executed? Fill out the right side of the template. Submit the appropriate command-line parameter together with the filled-out template.

Note: On an x86 processor, the byte order is little-endian.



b) Injecting Code

A buffer overflow not only allows altering a program’s control flow, but also allows injecting malicious code. Assume that the buffer `padded_passwords` is located at memory address `0xC0DEBEEF` during execution. What could a possible command-line parameter with an exploit look like? Assume you want to copy and execute the following 21-byte dummy shellcode in the buffer.

```
<SHELLCODE> = "\xe1\x69\x89\x2f\x80\x0b\x68\x68\xf7\xcd\x73\x31\xb0\x2f\x68\xe3\x51\xc9\x2f\x6e\x62"
```

Briefly explain why it would be problematic if `padded_password` was located at `0xC0DE007A` instead of `0xC0DEBEEF`.

c) Countermeasure

How could the source code be changed to effectively prevent the vulnerability? Explain in detail.

Plagiarism & Use of Generative AI

In this course, we respect other people's work and do not tolerate plagiarism. That means you should always give credit to the original authors if you:

- use someone else's work (whether you copy it directly, almost directly, or rewrite it in your own words),
- reuse parts of someone else's work, such as equations, figures, or tables.

We strongly encourage you to work on this assignment without using generative AI tools (like large language models), so you can build your own understanding of the material. Please, **in all cases**, state clearly how you used such tools:

- If you only used them for small edits (e.g., grammar or wording), add a note like: "This assignment was edited for grammar using [Tool Name]."
- If you did not use any AI tools, add: "This assignment was completed without the use of AI tools."
- If you use AI for more than small edits, please specify in detail which parts of the solutions are generated by the AI tool and which steps you took to ensure the correctness of the output.
- Work that relies on AI to generate the major part of the submission, e.g., full scripts or key parts of the solution, will not be accepted.

If you use Stack Overflow or similar resources, make sure to clearly cite any code you did not write yourself.